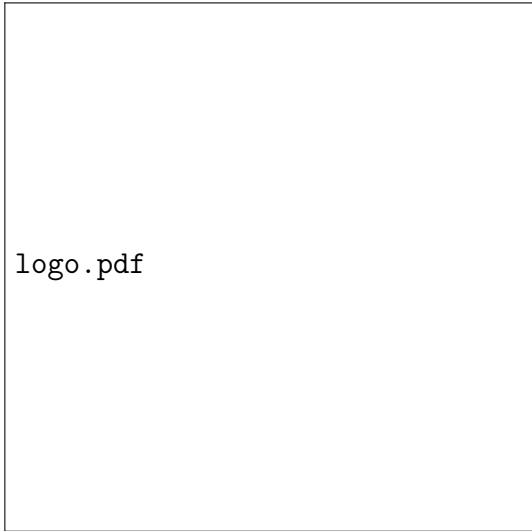




logo.pdf

TEORÍA DE ALGORITMOS
CURSO ECHEVARRIA

Trabajo Práctico X

April 9, 2025

NOMBRE - PADRON

Índice

1	Supuestos	3
1.0.1	Ejercicio2	3
2	Diseño	4
2.0.1	Ejercicio2	4
3	Análisis de complejidad	5
3.0.1	Ejercicio2	5
4	Seguimiento de Ejemplo	6
4.1	Ejercicio 2	6
5	Casos de prueba y mediciones	6
6	Resultados	7
6.1	Ejercicio 2	7
7	Referencias	7
7.1	Ejercicio 2	7
8	Generacion de sets	7
8.1	Ejercicio 2	7

1 Supuestos

1.0.1 Ejercicio2

Se supone que las distancias se encuentran en una lista, donde cada posicion es la distancia a la anterior posicion, y cada posicion representa una parada, el resultado se genera almacenando los subindices de cada parada.

2 Diseño

2.0.1 Ejercicio2

- En el main, se generan los archivos que almacenan las distancias a evaluar con una semilla fija, luego se generan los archivos donde van los resultados y se llama a la funcion que realiza el calculo.
- La funcion generar_datos, genera una lista con datos aleatorios basandose en la semilla recibida, estos datos son numeros enteros de 0 a 7 inclusive y se genera una lista de largo recibido que es retornada.
- La funcion calcular_mejor_distancia recorre mientras que el elemento que estoy evaluando es menor que el largo de la lista de paradas, y luego avanza mientras que la suma de distancias sea menor que 7, modificando la posicion actual y luego una vez que la suma exceda 7 se agrega a la lista de resultados el indice -1 que seria la posicion que cumple el rango, luego continua siguiendo esta misma idea, y agrega el ultimo paraje para llegar hasta el final, imprime el tiempo que tardo y devuelve una lista con los resultados.

3 Analisis de complejidad

3.0.1 Ejercicio2

- La parte de generar los datos no la tengo en cuenta en el analisis, solo la parte de calcular los resultados, time es $O(1)$, inicializar variables tambien es $O(1)$, len(distancias), es $O(1)$, imprimir y utilizar append es $O(1)$, los while anidados, comparten la condicion de que la posicion sea menor que el largo y en cada iteracion se avanza al menos una vez, por lo que la complejidad de esta parte es $O(n)$, como conclusion, el resultado es $O(n)$.

4 Seguimiento de Ejemplo

4.1 Ejercicio 2

- Vamos a hacer un ejemplo con la lista de distancias = [4, 3, 6, 6, 1, 7], Al llegar a la funcion se inicializan las variables, y el largo queda en 5, se entra al primer while ya que i esta en 0 que es menor que el largo, luego se entra al otro while, repitiendo la condicion y evaluando que el elemento en la lista en la posicion i (0) sea menor o igual que 7, como es el caso (4), se entra al while y se suma el actual en auxiliar(4) y se aumenta la posicion(1), como ahora la posicion es 1 y la suma en auxiliar es 4, evaluando la condicion en auxiliar + el valor en la nueva posicion(1) (valor = 3), como auxiliar(7) es menor o igual que 7, se suma este valor parcial en auxiliar(7) y se suma en 1 la posicion(2), ahora al evaluar la condicion de menor o igual que 7 va a dar false porque el valor es 13 que es mayor, se sale de este while, se guarda el indice de la posicion actual a la anterior (2-1), y se continua iterando de esta forma hasta llegar a la ultima posicion (5), donde esta posicion ya no es menor que el largo por lo que se deja de iterar y se agrega este indice, se calcula e imprime el tiempo final. Devuelve una lista con los resultados obtenidos.

5 Casos de prueba y mediciones

6 Resultados

6.1 Ejercicio 2

- Los resultados para 1000, 10000, 100000, 500000 y 1000000, son insertar resultados, respectivamente, por lo que si el resultado esperado coincide con la complejidad analizada $O(n)$

7 Referencias

7.1 Ejercicio 2

- Van Rossum, G., & Python Software Foundation. (2023). Built-in Functions: open. Retrieved from <https://docs.python.org/3/library/functions.html#open>
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

8 Generacion de sets

8.1 Ejercicio 2

- para generar los sets se realiza de la siguiente manera, utilizando 1 como semilla para el largo 1000, 2 como semilla para el largo 100000, 3, como semilla para largo 100000, 4 como semilla para largo 500000, y 5 como semilla para largo 1000000.

```
1 def generar_datos(largo, semilla):  
2     random.seed(semilla)  
3     return [random.randint(0, MAXIMA_DISTANCIA) for _ in range(  
    largo)]
```